Technology

Building a robust frontend using progressive enhancement

Give feedback about this page (/contact/govuk)

From: **Technology community (frontend development)**

For users to experience a quality service it must be built in a robust way.

Progressive enhancement is a way of building websites and applications based on the idea that you should make your page work with HTML first.

Only after this can you add anything else like Cascading Style Sheets (CSS) and JavaScript.

All government services must follow progressive enhancement, even if part of the service or a parent service needs JavaScript

Building your service using progressive enhancement will:

- ensure your service is robust and of a high quality
- make it more likely your service will work regardless of which browser or device is being used (https://www.gov.uk/service-manual/technology/designing-for-different-browsers-and-devices)
- mean your service's most basic functionality will work and meet the core needs of the user
- improve accessibility by encouraging best practices like writing semantic markup (https://html.com/semantic-markup/)

Contents

— Start with HTML

— Using CSS

— Using JavaScript

— Single page applications

— Testing your service

— Case studies and related guides

connectivity limitations to use

rcial off the shelf' (COTS)
nal supplier to build the
ler including this in your

requirements as part of your purchasing strategy (https://www.gov.uk/guidance/define-your-purchasing-strategy).

Start with HTML

Government services should be functional using only HTML. This includes services such as:

- transactional services, for example forms that let the user provide information to government
- smart answers (https://insidegovuk.blog.gov.uk/2016/10/27/making-it-easier-to-understand-smart-answer-logic/), for example the Registering a birth abroad (https://www.gov.uk/register-a-birth) service
- content-based websites, for example GOV.UK's foreign travel advice (https://www.gov.uk/foreign-travel-advice) page

The HTML layer is fault-tolerant by design. The browser ignores markup that it does not understand and continues to parse it as best as it can.

This means that older browsers will be very likely to be able to load the HTML for your service, even if there are bugs in the HTML or you use features that only exist in more modern browsers.

A user should be able to navigate through your service using only the HTML. You should use correct semantic markup and structure your source order and document outline (https://www.w3.org/WAI/tutorials/page-structure/) logically.

Using CSS

You can style your service using CSS to make it look like GOV.UK.

The CSS layer is reasonably fault-tolerant. This means the browser will ignore individual declarations that it does not understand, for example if you use a property that only exists in newer browsers.

Be aware that the browser will ignore the entire ruleset if it encounters a selector that it does not understand.

There are also other reasons why the CSS may fail to load.

> Avoid techniques such as 'CSS-in-JS' to ensure that your site still looks correct even if the JavaScript fails to load.

Using JavaScript

JavaScript can be used to add interactive elements to your service.

The JavaScript layer is not fault-tolerant. If your JavaScript uses a syntax or calls an API that is not supported in the user's browser, it will error and the rest of the JavaScript will not run.

There are other reasons why the JavaScript may fail to load or run.

If your service is not designed with this in mind, users may be unable to use it.

You can increase the chances that your JavaScript will work correctly in all browsers by:

- using feature detection (https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Cross_browser_testing/Feature_detection) for browser APIs
- including polyfills (https://developer.mozilla.org/en-US/docs/Glossary/Polyfill) for newer browser features
- transpiling your JavaScript to a common syntax that your target browsers can understand
- using automated tests or linters

- doing regular manual testing, including testing with older or lower-powered devices

Transpilation and polyfills can significantly increase the size of your JavaScript. You should consider the trade offs involved and revisit these decisions regularly as browser usage changes.

> Where possible the JavaScript should enhance HTML and CSS that provides the same core functionality. For example, an autocomplete could enhance a < select > element, or something similar. This still lets the user do what they need to do, even if the JavaScript fails.

Alternatives to JavaScript

If you believe your service can only be built using JavaScript, you should think about using simpler solutions that are built using HTML and CSS and will meet user needs.

For example, if you want to use JavaScript to provide interactive graphs, other options are to:

- display the data in a table
- allow the data to be exported so that it can analysed in another application
- pre-render the graphs as images

If the core functionality of your service cannot be provided without JavaScript, you'll need to consider how users can access your service through other channels. This might be telephone calls or in-person visits to offices.

Using client-side JavaScript frameworks

If your service is mostly built using components from the GOV.UK Design System (https://design-system.service.gov.uk/) and doesn't have a complex user interface, you do not need to use a client-side JavaScript framework.

The components in the GOV.UK Design System (https://design-system.service.gov.uk/) include how to import

JavaScript (https://frontend.design-system.service.gov.uk/import-javascript) to your service without the need for a framework.

If you do choose to use client-side JavaScript frameworks (https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks), be aware that although they can be helpful when building a service with a complex user interface, they can introduce problems.

Using a client-side JavaScript framework can:

- increase the overall size of your code base and push processing to the client-side, causing performance issues for users with a slower network connection or lower powered device
- create a reliance on third-party code that your developers do not have control over, requiring you to make major changes to your service in order to stay up to date with changes in the framework
- make it difficult to find people with the skills required to maintain the code, if frameworks lose popularity over time

If you use a JavaScript framework you should:

- be able to justify with evidence, how using JavaScript would benefit users
- be aware of any negative impacts and be able to mitigate them
- consider whether the benefits of using it outweigh the potential problems
- only use the framework for parts of the user interface that cannot be built using HTML and CSS alone
- design each part of the user interface as a separate component

Having separate components means that if the JavaScript fails to load, it will only be that single component that fails. The rest of the page will load as normal.

> If you use JavaScript, it should only be used to enhance the HTML and CSS so users can still use the service if the JavaScript fails.

Reasons why CSS or JavaScript may fail to load or run

CSS and JavaScript can fail to load or run because of, for example:

- temporary network errors
- third-party browser extensions like ad blockers
- third-party supplier downtime, such as when using a content delivery network
- DNS lookup failures
- bugs introduced by browser updates
- bugs introduced in third party JavaScript intentionally running on the same page
- corporate firewalls blocking, removing or changing content (large institutions like banks or government departments may use these)
- mobile network providers changing content to speed up load times and reduce bandwidth usage
- personal firewalls or antivirus software changing or blocking content
- the use of unsecure connections, where internet providers insert their own code into the page that accidentally conflicts with your own

> Some users may deliberately turn off features in their browsers. You should respect their decision and make sure those users can still use your service.

Single page applications

Do not build your service as a single-page application (SPA). This is where the loading of pages within your service is handled by JavaScript, rather than the browser.

Single page applications rarely bring benefits and can make the service inaccessible because:

- users of assistive technology would be unaware of changes in context, for example when moving to a new page

- it would fail to handle focus when moving between pages
- the user would be unable to navigate using the back or forward buttons in their browser
- users would be unable to recover from an error, for example if there is an interruption to their network connection

Testing your service

If any components in your service rely heavily on JavaScript or JavaScript frameworks you will need to make sure they:

- work on a wide range of browsers and devices
- work with assistive technologies (https://www.gov.uk/service-manual/technology/testing-with-assistive-technologies)
- are tested to ensure good performance (https://www.gov.uk/service-manual/technology/how-to-test-frontend-performance)

> Ideally you should check that these components work in your service by testing it for accessibility (https://www.gov.uk/service-manual/helping-people-to-use-your-service/testing-for-accessibility).

Case studies and related guides

- Why we use progressive enhancement to build GOV.UK (https://gdstechnology.blog.gov.uk/2016/09/19/why-we-use-progressive-enhancement-to-build-gov-uk/)
- Designing for different devices (https://www.gov.uk/service-manual/technology/designing-for-different-browsers-and-devices)
- How to test for front end performance (https://www.gov.uk/service-manual/technology/how-to-test-frontend-performance)
- Understanding WCAG 2.2 (https://www.gov.uk/service-manual/helping-people-to-use-your-service/understanding-wcag)

Published 23 May 2016
Last updated 27 September 2024 + show all updates

OGL All content is available under the Open Government Licence v3.0, except where otherwise stated

© Crown copyright